

# A Tiny Vision-Language Model

AUTHOR

Dhia Sarraj, University of Monastir, ISIMM

## I. PROBLEM

The challenge is to train a **vision-language model** (VLM) that generates a character-level description of an image. Each input is a 64×64 RGB image containing one or two objects, and the target word is a scene description written as **one word**, with no spaces. The model uses a CNN to extract visual features, transform them into visual tokens, and generates the description using a Transformer decoder with self-attention. The main objective of this challenge was to implement a VLM from scratch, including the visual encoder, the transformer decoder and the self attention.

## II. ARCHITECTURE

The proposed model follows an encoder-decoder architecture adapted for vision models. The encoder consists of the concatenation of two layers of convolutional neural networks that maps the input image to a feature representation, which is projected into a sequence of visual tokens. These tokens are concatenated with the embedded text tokens and provided to the decoder. The overall architecture is shown in Figure 1.

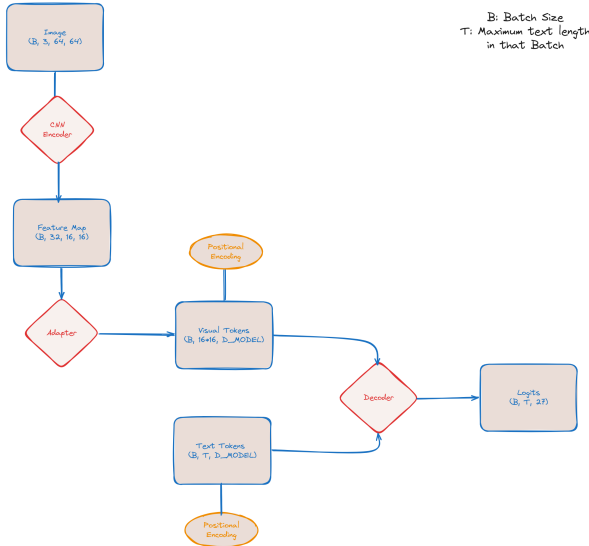


Figure 1: VLM - Model Architecture.

At each step, the model predicts the next character from the previously generated characters and the visual representation.

## III. IMPLEMENTATION

### I. Attention

The Transformer attention mechanism (inspired by **Attention Is All You Need** paper ) was implemented from scratch without the use of a built-in multi-head attention modules.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \quad (1)$$

The implementation uses multiple attention heads (4 in this architecture), in order to allow the model to focus on different features in parallel before combining their outputs.

### II. Generation

During inference, my model generates one character at a time. At each step, the last generated tokens are provided to the decoder, which predicts the next token. The decoder selects the token with the highest predicted probability at each step, and the generation stops when the EOS token is produced or when the maximum sequence length is reached (max\_length = 45 letters).

## IV. TRAINING SETUP

**Table 1:** Training and model parameters.

| Parameter           | Value              |
|---------------------|--------------------|
| Batch size          | 32                 |
| Learning rate       | $1 \times 10^{-2}$ |
| Epochs              | 16                 |
| Optimizer           | AdamW              |
| $D_{\text{MODEL}}$  | 128                |
| Number of heads     | 4                  |
| Number of layers    | 6                  |
| CNN filters         | 32                 |
| Kernel size         | 3                  |
| Maximum text length | 46                 |
| Vocabulary size     | 27                 |
| Seed                | 42                 |

## V. EXPERIMENTS

During training the model, I was particularly curious about how changing the **number of attention heads** and **learning rate** would affect the model’s word generation. So I focused on these two hyperparameters.

### I. Experiment 1 — Learning Rate

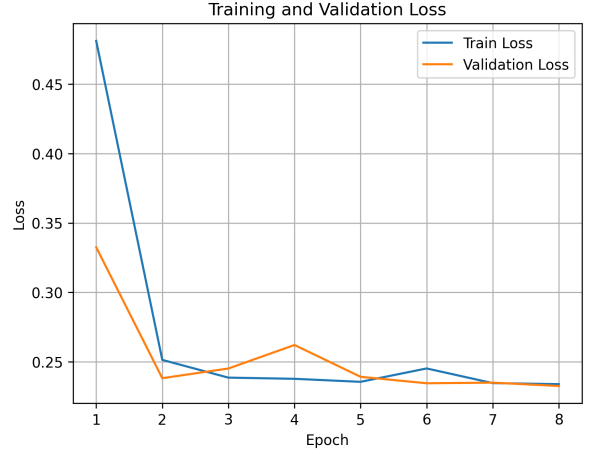
**Question:** I wanted to see how changing the learning rate would affect the model.

**Hypothesis:** I expected that increasing the learning rate will make the model learn quickly. A learning rate that is too small may make training slower, while a learning rate that is too large may make training unstable.

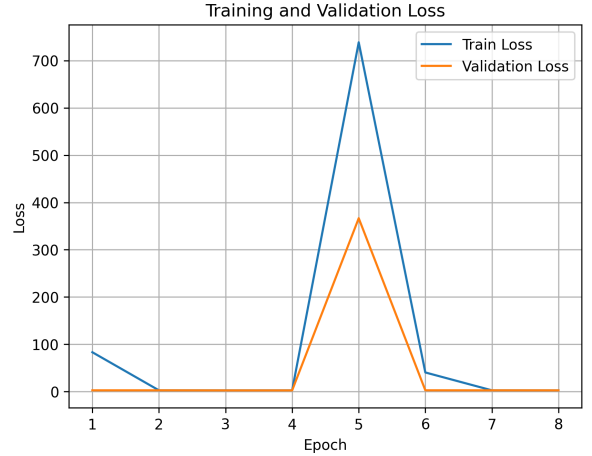
**Changed:** Only the learning rate was varied while keep the other hyperparameters fixed. The baseline value was  $1 \times 10^{-2}$ , with additional experiments using smaller and larger values.

**Table 2:** Benchmark of model learning rate.

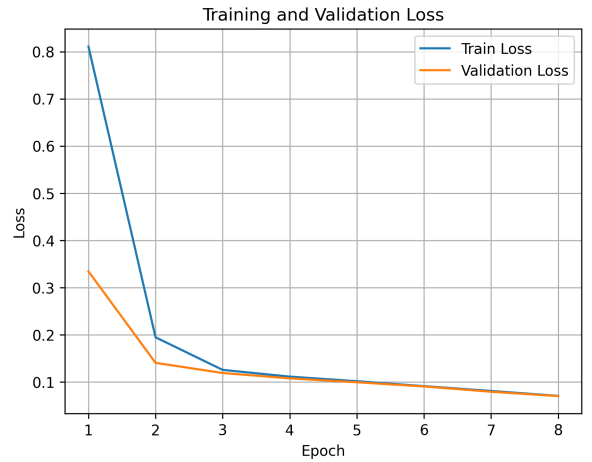
| Model      | Learning Rate      | Train loss | Exact match |
|------------|--------------------|------------|-------------|
| Baseline   | $1 \times 10^{-2}$ | 0.2339     | 0.2325      |
| Larger LR  | 1                  | 2.4543     | 2.5131      |
| Smaller LR | $1 \times 10^{-4}$ | 0.0707     | 0.0702      |



**Figure 2:** Loss Curves with  $LR = 1 \times 10^{-2}$



**Figure 3:** Loss Curves with  $LR = 1$



**Figure 4:** Loss Curves with  $LR = 1 \times 10^{-4}$

## II. Experiment 2 — Number of Attention Heads

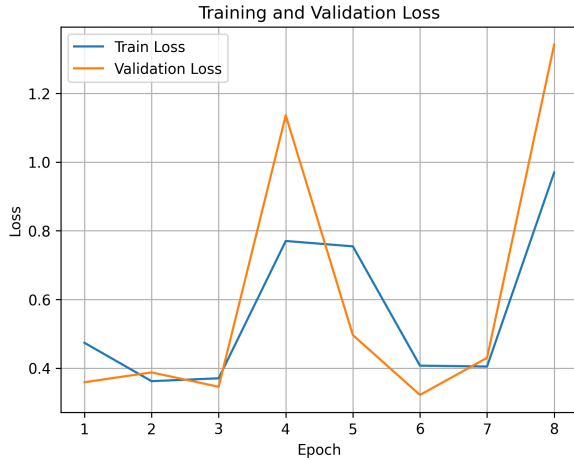
**Question:** I was also curious about whether using more attention heads would help the model understand better the image.

**Hypothesis:** I expected multiple attention heads will allow the model focusing on different shapes in the image, and therefore improve the generation performance.

**Changed:** The number of attention heads was varied while keep the other hyperparameters fixed. The baseline value was 4, with additional experiments using larger values.

**Table 3:** Benchmark of attention heads .

| Model       | Nb of Heads | Train loss | Exact match |
|-------------|-------------|------------|-------------|
| Baseline    | 4           | 0.2339     | 0.2325      |
| More Heads  | 8           | 0.9696     | 1.3420      |
| More+ Heads | 16          | 0.2369     | 0.2322      |



**Figure 5:** Loss Curves with Nb\_Heads = 8



**Figure 6:** Loss Curves with Nb\_Heads = 16

## VI. LIMITATIONS

One of the main limitations I faced was the limited GPU resources available on Kaggle, which made it difficult to test more different model configurations in a small period of time. I also used only one random seed, so some of the results may vary if the experiments are repeated with different seeds. In the future, I would like to do a benchmark with multiple seeds and see the results. I would also like to test the model on a different image dataset, and as a result, I believe that I will have to scale my model in order to focus on more shapes and features.